TRABALHO DE PÓS-GRADUAÇÃO

Cybersecurity Offensive: Pentest & Red Team

Lab 1: Attack Surface Mapping

Mapeamento de superfície de ataque na aplicação OWASP Juice Shop com Burp Suite

Aluno: Victor Caldas dos Santos

1. Objetivo

Este trabalho tem como objetivo realizar o mapeamento da superfície de ataque (attack surface mapping) da aplicação vulnerável OWASP Juice Shop, por meio da captura e análise do tráfego HTTP gerado durante a navegação manual pela aplicação, utilizando o Burp Suite Community Edition como proxy de interceptação.

A partir da captura de tráfego, buscou-se identificar endpoints, objetos, parâmetros controlados pelo cliente e fluxos de dados sensíveis, de forma a construir uma visão inicial dos possíveis vetores de ataque da aplicação.

2. Metodologia

- Ambiente: aplicação OWASP Juice Shop executada localmente (http://127.0.0.1:3000).
- Ferramenta de interceptação: Burp Suite Community Edition v2026.2.3, configurado como proxy do navegador.
- Técnica: navegação manual e exploratória por todas as áreas da aplicação (cadastro, login, produtos, carrinho, checkout, perfil, endereços, cartões, reclamações, feedbacks, perguntas/respostas de segurança), permitindo que o tráfego fosse registrado passivamente na aba Proxy → HTTP History.
- Critério de seleção: para cada requisição analisada, foram registrados o endpoint, o método HTTP, ao menos um parâmetro controlado pelo cliente (query string, path ou body) e o status code da resposta.
- Resultado: identificação de mais de 20 endpoints distintos sob o caminho /api e /rest, dos quais 12 foram selecionados para compor este relatório, priorizando variedade de métodos, status codes e relevância dos dados trafegados.

3. Evidência geral do tráfego capturado

A imagem abaixo apresenta o Site Map do Burp Suite, evidenciando a estrutura de endpoints identificada sob o caminho /api durante a navegação pela aplicação:

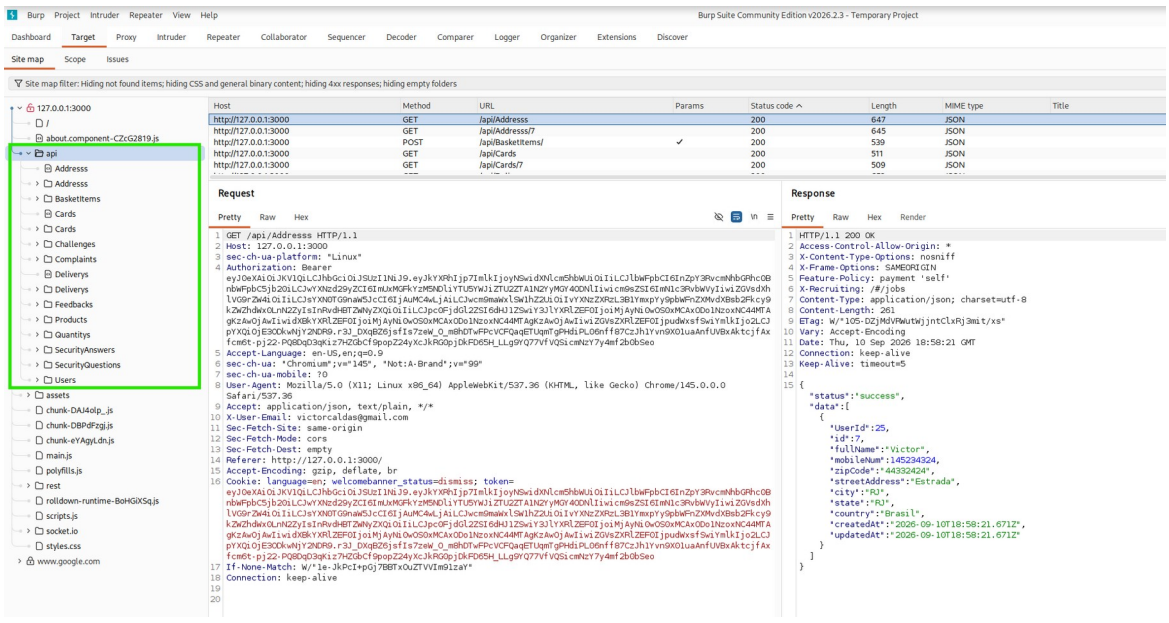


Figura 1. Site Map do Burp Suite com os endpoints identificados sob /api.

A imagem a seguir apresenta um trecho do HTTP History do Burp, evidenciando o volume e a variedade de requisições capturadas sob o caminho /rest, incluindo endpoints administrativos, de carrinho, busca, autenticação e carteira digital:

14	http://127.0.0.1:3000	GET	/rest/admin/application-configuration		304	306	
17	http://127.0.0.1:3000	GET	/rest/admin/application-configuration		304	306	
19	http://127.0.0.1:3000	GET	/rest/admin/application-configuration		304	306	
25	http://127.0.0.1:3000	GET	/rest/admin/application-configuration		304	306	
55	http://127.0.0.1:3000	GET	/rest/admin/application-configuration		304	306	
80	http://127.0.0.1:3000	GET	/rest/admin/application-configuration		304	306	
88	http://127.0.0.1:3000	GET	/rest/admin/application-configuration		304	306	
92	http://127.0.0.1:3000	GET	/rest/admin/application-configuration		304	306	
138	http://127.0.0.1:3000	GET	/rest/admin/application-configuration		304	306	
152	http://127.0.0.1:3000	GET	/rest/admin/application-configuration		304	306	
181	http://127.0.0.1:3000	GET	/rest/admin/application-configuration		304	306	
185	http://127.0.0.1:3000	GET	/rest/admin/application-configuration		304	306	
197	http://127.0.0.1:3000	GET	/rest/admin/application-configuration		304	306	
13	http://127.0.0.1:3000	GET	/rest/admin/application-version		304	304	
18	http://127.0.0.1:3000	GET	/rest/admin/application-version		304	304	
96	http://127.0.0.1:3000	GET	/rest/basket/6		200	539	JSON
98	http://127.0.0.1:3000	GET	/rest/basket/6		200	909	JSON
103	http://127.0.0.1:3000	GET	/rest/basket/6		304	305	
125	http://127.0.0.1:3000	GET	/rest/basket/6		304	305	
130	http://127.0.0.1:3000	GET	/rest/basket/6		304	305	
143	http://127.0.0.1:3000	GET	/rest/basket/6		304	305	
149	http://127.0.0.1:3000	GET	/rest/basket/6		200	539	JSON
148	http://127.0.0.1:3000	POST	/rest/basket/6/checkout	✓	200	429	JSON
175	http://127.0.0.1:3000	GET	/rest/captcha/		200	431	JSON
201	http://127.0.0.1:3000	GET	/rest/captcha/		200	432	JSON
198	http://127.0.0.1:3000	GET	/rest/deluxe-membership		200	433	JSON
15	http://127.0.0.1:3000	GET	/rest/languages		304	306	
184	http://127.0.0.1:3000	GET	/rest/memories/		200	6571	JSON
51	http://127.0.0.1:3000	GET	/rest/products/1/reviews		200	781	JSON
53	http://127.0.0.1:3000	GET	/rest/products/1/reviews		304	305	
54	http://127.0.0.1:3000	GET	/rest/products/1/reviews		304	305	
20	http://127.0.0.1:3000	GET	/rest/products/search?q=	✓	304	306	
56	http://127.0.0.1:3000	GET	/rest/products/search?q=	✓	304	306	
84	http://127.0.0.1:3000	GET	/rest/products/search?q=	✓	304	306	
102	http://127.0.0.1:3000	GET	/rest/products/search?q=	✓	304	306	
154	http://127.0.0.1:3000	GET	/rest/products/search?q=	✓	304	306	
202	http://127.0.0.1:3000	GET	/rest/products/search?q=	✓	304	306	
150	http://127.0.0.1:3000	GET	/rest/track-order/5b9d-569118233da04adc		200	744	JSON
93	http://127.0.0.1:3000	POST	/rest/user/login	✓	200	1191	JSON
95	http://127.0.0.1:3000	GET	/rest/user/whoami		200	394	JSON
100	http://127.0.0.1:3000	GET	/rest/user/whoami		200	519	JSON
52	http://127.0.0.1:3000	GET	/rest/user/whoami?fields=email	✓	200	394	JSON
94	http://127.0.0.1:3000	GET	/rest/user/whoami?fields=email	✓	304	303	
99	http://127.0.0.1:3000	GET	/rest/user/whoami?fields=email	✓	200	427	JSON
126	http://127.0.0.1:3000	GET	/rest/user/whoami?fields=email	✓	304	304	
131	http://127.0.0.1:3000	GET	/rest/user/whoami?fields=email	✓	304	304	
145	http://127.0.0.1:3000	GET	/rest/user/whoami?fields=email	✓	304	304	
176	http://127.0.0.1:3000	GET	/rest/user/whoami?fields=id,email	✓	200	435	JSON
177	http://127.0.0.1:3000	GET	/rest/user/whoami?fields=id,email	✓	304	304	
179	http://127.0.0.1:3000	GET	/rest/user/whoami?fields=id,email	✓	304	304	
200	http://127.0.0.1:3000	GET	/rest/user/whoami?fields=id,email	✓	304	304	
139	http://127.0.0.1:3000	GET	/rest/wallet/balance		200	413	JSON

Figura 2. HTTP History do Burp Suite com endpoints sob /rest.

4. Mapa de endpoints (mínimo de 12)

A tabela a seguir consolida os 12 endpoints selecionados para este relatório, com método HTTP, status code da resposta e ao menos um parâmetro controlado pelo cliente identificado em cada requisição:

#	Endpoint	Método	Status	Parâmetro controlado pelo cliente
1	/api/Addresss/	**POST**	**201**	fullName, mobileNum, zipCode, streetAddress, city, state, country (body)
2	/api/BasketItems/	**POST**	**200**	ProductId, BasketId, quantity (body)
3	/api/Cards/	**POST**	**201**	fullName, cardNum, expMonth, expYear (body)
4	/api/Challenges/?name=Score Board	**GET**	**304**	name (query string)
5	/api/Complaints/	**POST**	**201**	message (body)
6	/api/Deliverys/1	**GET**	**200**	id (path)
7	/api/Feedbacks/	**GET**	**200**	Nenhum (endpoint público, sem parâmetro)
8	/api/Products/1?d=...	**GET**	**304**	id (path) + d (query string)
9	/api/Quantitys/	**GET**	**200**	Nenhum (endpoint público, sem parâmetro)
10	/api/SecurityAnswers/	**POST**	**201**	UserId, answer, SecurityQuestionId (body)
11	/api/SecurityQuestions/	**GET**	**200**	Nenhum (endpoint público, sem parâmetro)
12	/api/Users/	**POST**	**201**	email, password, passwordRepeat, securityQuestion, securityAnswer (body)

5. Capturas e anotações por endpoint

Nesta seção são apresentadas as capturas do Burp Suite (Request/Response) referentes a cada um dos 12 endpoints mapeados, acompanhadas de anotação sobre o parâmetro controlado pelo cliente e observações relevantes.

5.1. /api/Addresss/

Método: POST **Status:** 201 Created

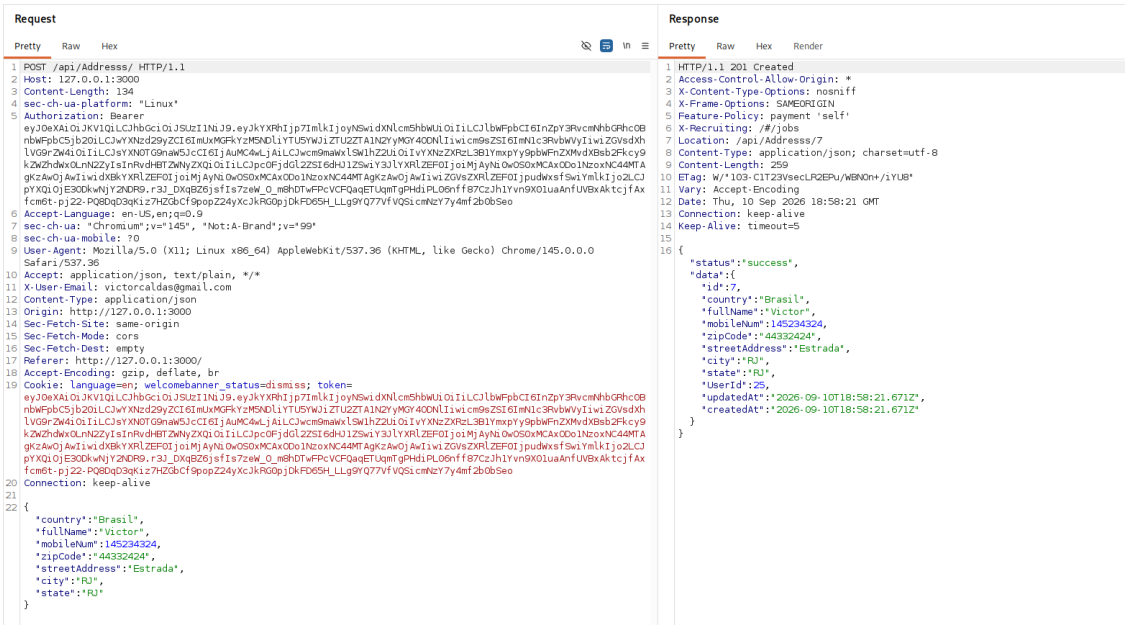


Figura 3. Request/Response capturado no Burp Suite para POST /api/Addresss/.

Cadastro de endereço. O body é totalmente controlado pelo cliente (fullName, mobileNum, zipCode, streetAddress, city, state, country), sem indícios visíveis de validação de formato (ex.: CEP, telefone).

5.2. /api/BasketItems/

Método: POST **Status:** 200 OK

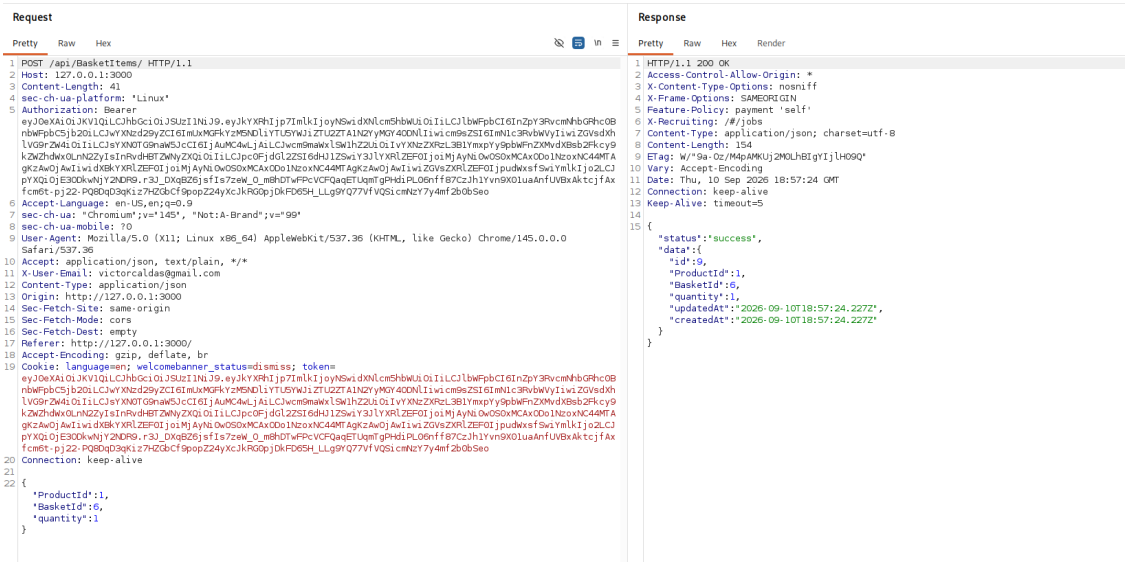


Figura 4. Request/Response capturado no Burp Suite para POST /api/BasketItems/.

Adição de item ao carrinho. O parâmetro BasketId é enviado pelo cliente no body. Se o backend não validar a titularidade do carrinho, isso é um vetor potencial de IDOR (inserir itens no carrinho de outro usuário).

5.3. /api/Cards/

Método: POST **Status:** 201 Created

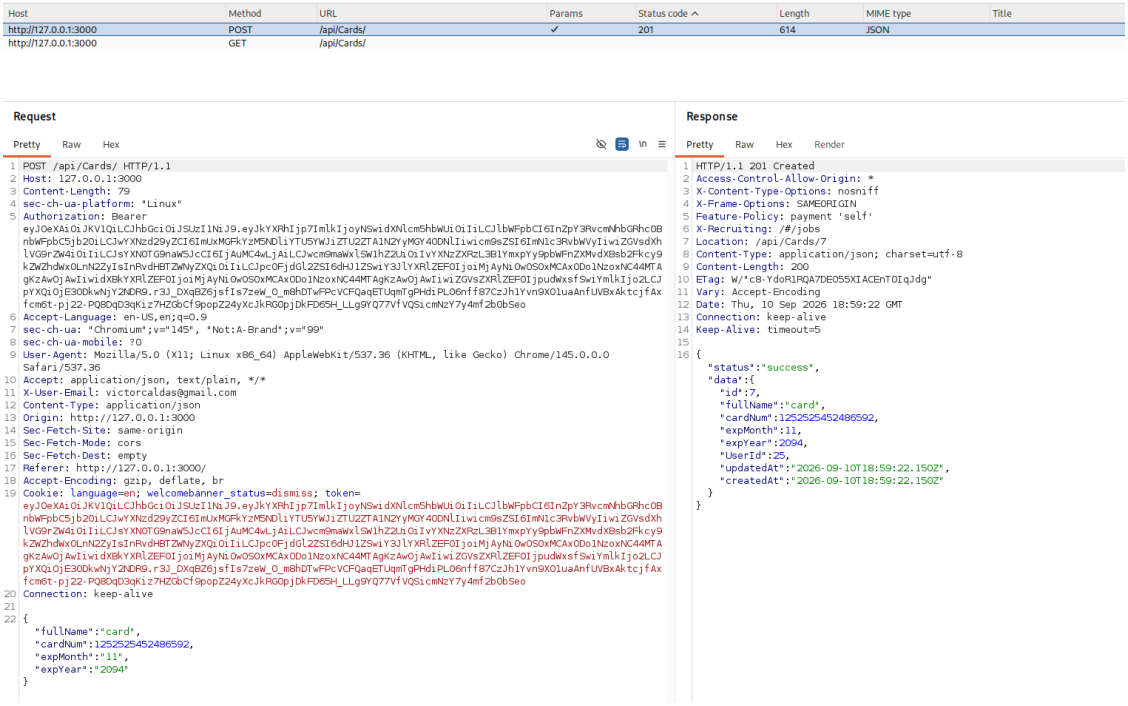


Figura 5. Request/Response capturado no Burp Suite para POST /api/Cards/.

Cadastro de cartão de pagamento. O número completo do cartão (cardNum) trafega em texto claro no body e é retornado sem mascaramento na resposta, caracterizando exposição de dado sensível de pagamento.

5.4. /api/Challenges/?name=Score Board

Método: GET **Status:** 304 Not Modified



Figura 6. Request/Response capturado no Burp Suite para GET /api/Challenges/?name=Score Board.

Consulta de challenge pelo parâmetro name na query string. Resposta em cache (304), validada via header If-None-Match/ETag.

5.5. /api/Complaints/

Método: POST **Status:** 201 Created

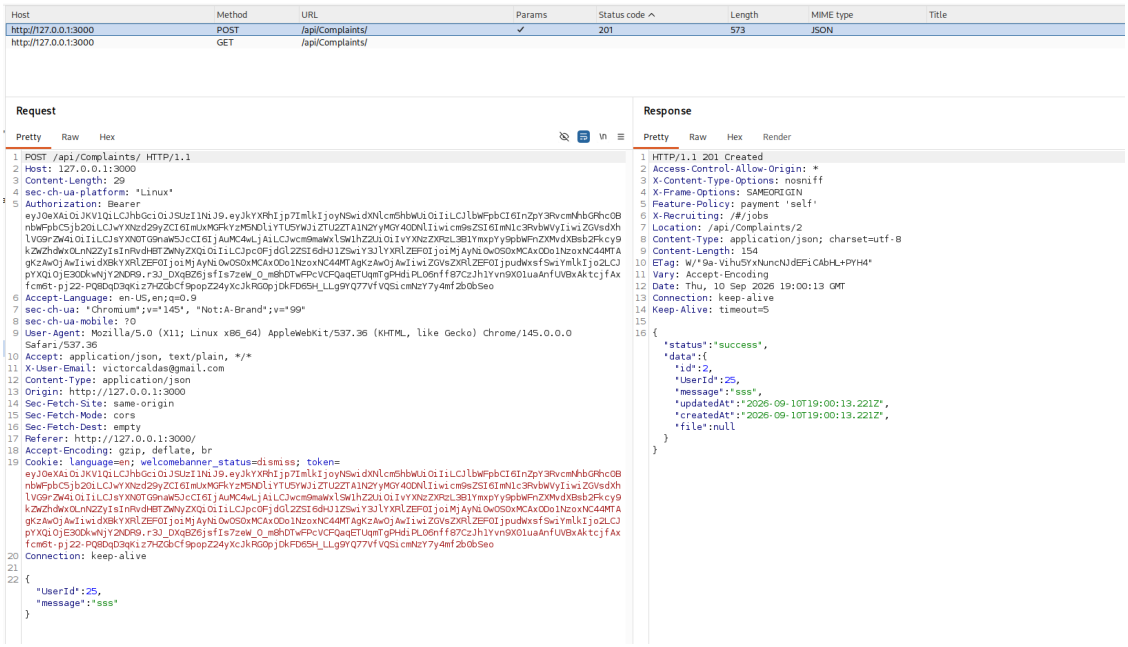


Figura 7. Request/Response capturado no Burp Suite para POST /api/Complaints/.

Registro de reclamação. O campo message é texto livre enviado pelo cliente, sem sanitização aparente, sendo candidato a testes de XSS e injeção.

5.6. /api/Deliverys/1

Método: GET **Status:** 200 OK

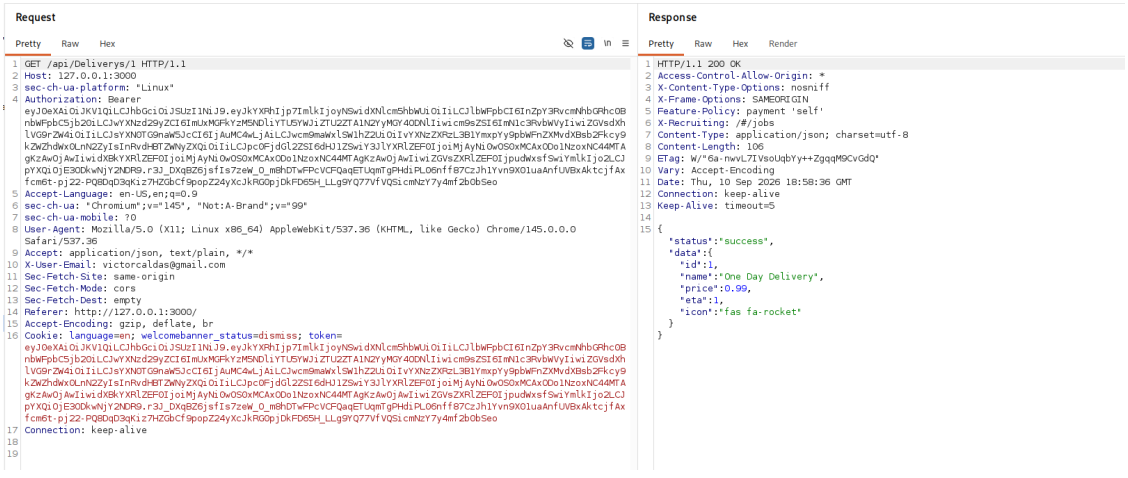


Figura 8. Request/Response capturado no Burp Suite para GET /api/Deliverys/1.

Consulta de opção de entrega pelo id no path. Mesmo padrão de referência direta a objeto (IDOR) observado em outros endpoints.

5.7. /api/Feedbacks/

Método: GET **Status:** 200 OK

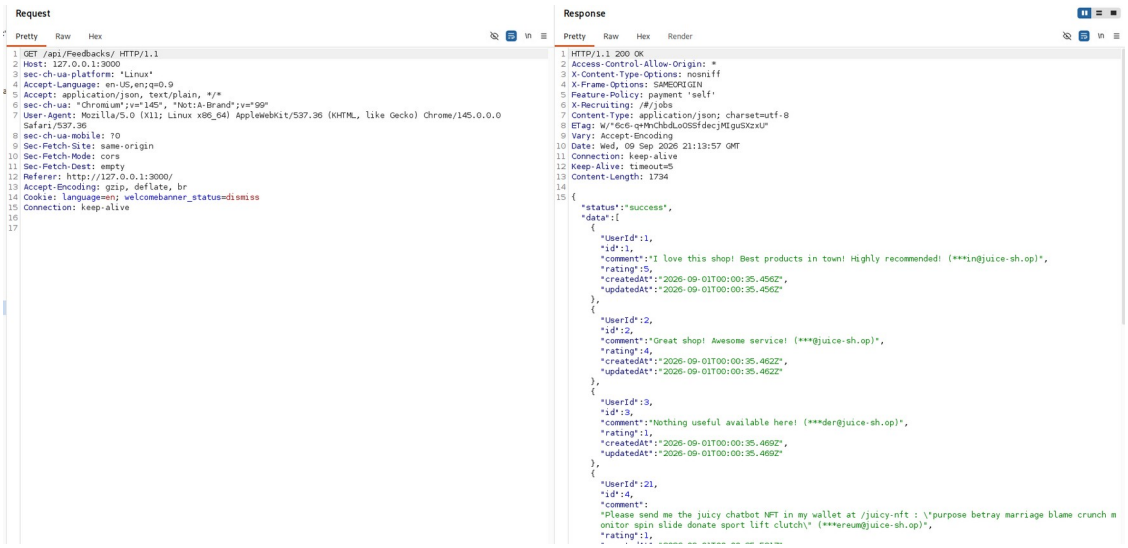


Figura 9. Request/Response capturado no Burp Suite para GET /api/Feedbacks/.

Endpoint público, sem header Authorization ou X-User-Email na requisição, que retorna comentários e o UserId de múltiplos usuários. Exposição de dado de terceiros sem necessidade de autenticação.

5.8. /api/Products/1?d=...

Método: GET **Status:** 304 Not Modified

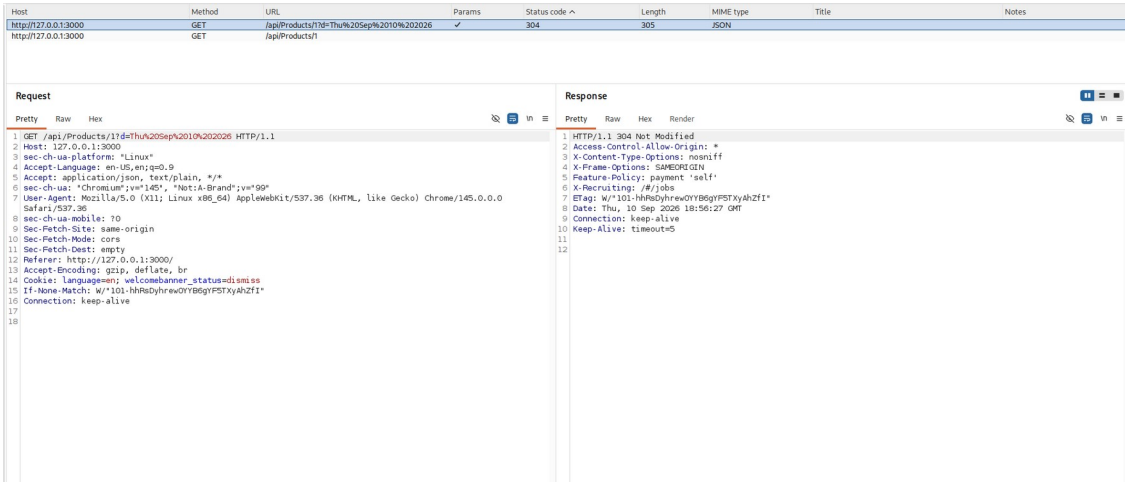


Figura 10. Request/Response capturado no Burp Suite para GET /api/Products/1?d=....

Consulta de produto por id no path, com parâmetro adicional d (data) na query string, aparentemente usado como cache-busting.

5.9. /api/Quantitys/

Método: GET **Status:** 200 OK

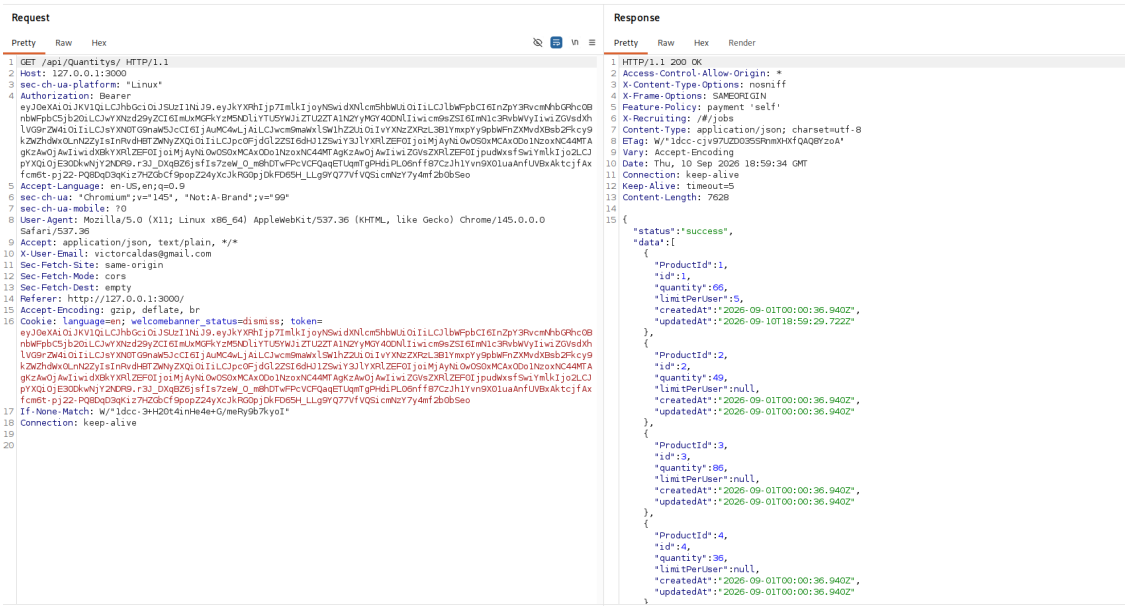


Figura 11. Request/Response capturado no Burp Suite para GET /api/Quantitys/.

Endpoint público que expõe dados internos de estoque (quantity, limitPerUser) de todos os produtos, sem exigir autenticação.

5.10. /api/SecurityAnswers/

Método: POST **Status:** 201 Created

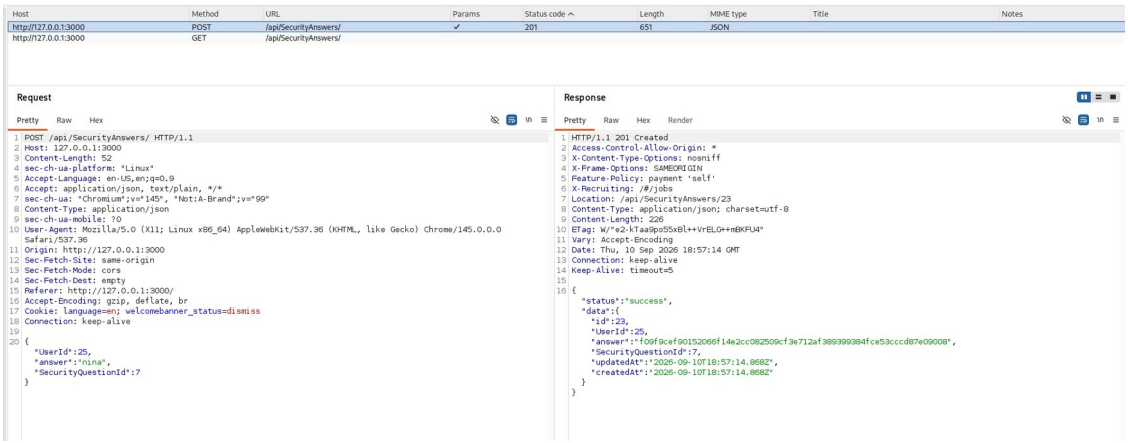


Figura 12. Request/Response capturado no Burp Suite para POST /api/SecurityAnswers/.

Cadastro de resposta de segurança. O campo UserId é enviado diretamente pelo cliente no body, o que é um possível vetor de Mass Assignment/IDOR, permitindo associar uma resposta a um UserId arbitrário caso não haja validação server-side vinculada ao token de sessão.

5.11. /api/SecurityQuestions/

Método: GET **Status:** 200 OK

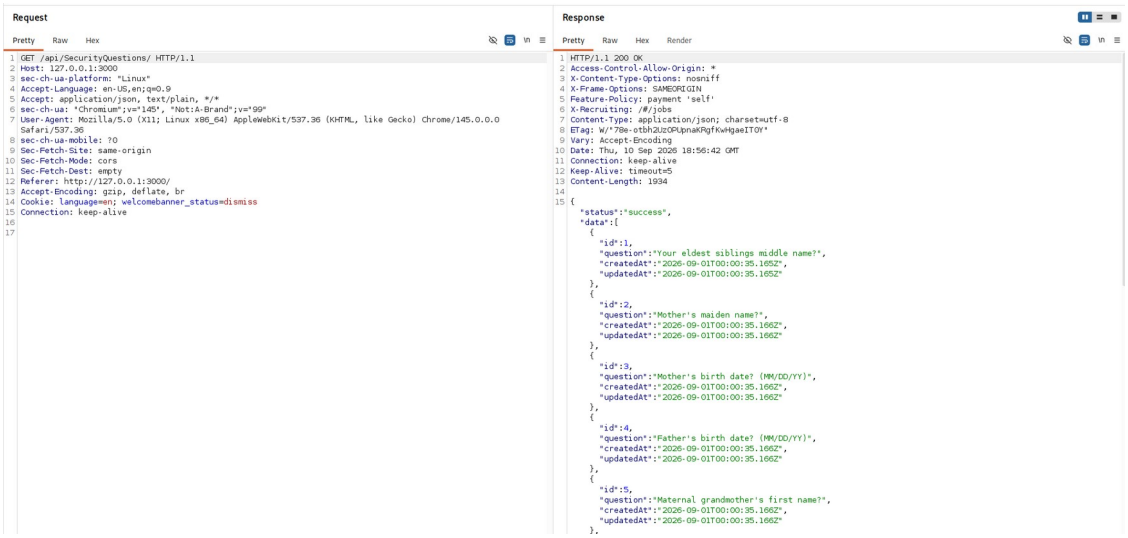


Figura 13. Request/Response capturado no Burp Suite para GET /api/SecurityQuestions/.

Endpoint público que lista todas as perguntas de segurança cadastradas no sistema, sendo útil para um atacante mapear o fluxo de recuperação de conta.

5.12. /api/Users/

Método: POST **Status:** 201 Created

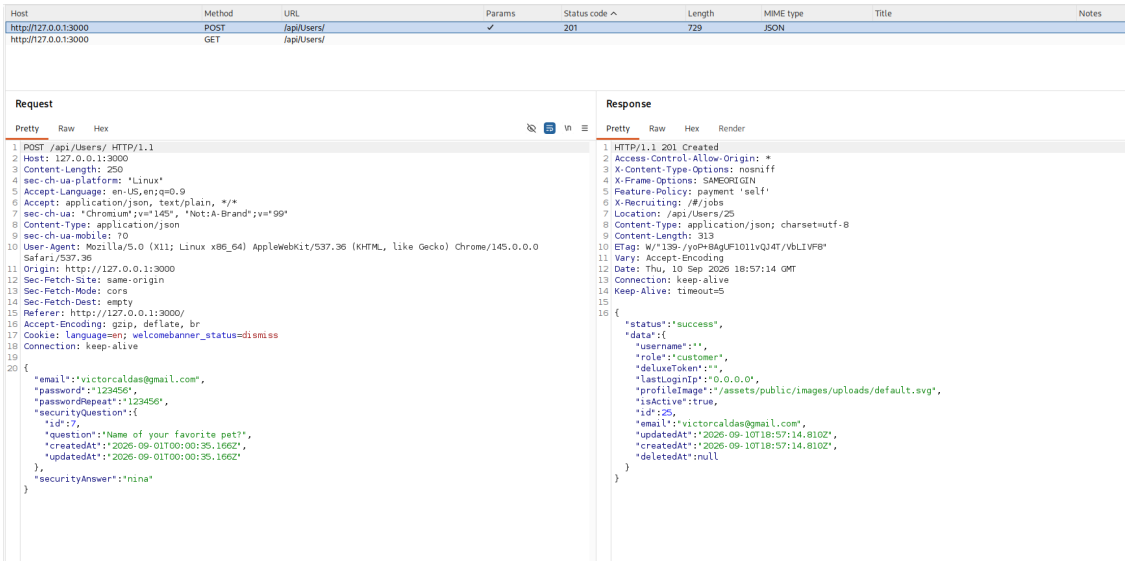


Figura 14. Request/Response capturado no Burp Suite para POST /api/Users/.

Registro de novo usuário. Os campos password e passwordRepeat trafegam em texto plano no body, junto com a resposta de segurança (securityAnswer), sendo o achado mais crítico do mapeamento.

6. Análise de fluxos sensíveis identificados

6.1. Exposição de dados pessoais (PII)

O endpoint /api/Addresss retorna dados pessoais completos do usuário (nome completo, telefone, CEP, endereço, cidade, estado e país) vinculados ao UserId, sendo a identificação feita por um header controlado pelo cliente (X-User-Email). A ausência de vínculo robusto entre o token de autenticação e o dado retornado é um ponto de atenção.

6.2. Exposição de dados de pagamento

O endpoint /api/Cards/ armazena e retorna o número completo do cartão de crédito (cardNum) sem qualquer mascaramento, junto com mês e ano de expiração. Trata-se de um comportamento inadequado do ponto de vista de proteção de dados de pagamento (PCI).

6.3. Credenciais em texto plano

No cadastro de usuário (/api/Users/) e no login (/rest/user/login), os campos password e passwordRepeat trafegam em texto plano no corpo da requisição. Embora o uso de HTTPS mitigue a exposição em trânsito, o fato reforça a importância de reforço de criptografia e políticas de senha no backend.

6.4. IDOR e Mass Assignment

Diversos endpoints utilizam identificadores sequenciais e previsíveis no path ou no body (ex.: /api/Addresss/{id}, /api/Cards/{id}, /api/Deliverys/{id}, BasketId em /api/BasketItems/, e UserId em /api/SecurityAnswers/), sugerindo possível vulnerabilidade de Insecure Direct Object Reference (IDOR) e Mass Assignment, ou seja, a possibilidade de um usuário manipular esses valores para acessar ou modificar dados de outros usuários.

6.5. Endpoints públicos expondo dados internos

Os endpoints /api/Feedbacks/, /api/Quantitys/ e /api/SecurityQuestions/ respondem sem exigir header de autenticação, expondo respectivamente comentários e UserId de terceiros, dados de estoque interno (quantity, limitPerUser) e a lista completa de perguntas de segurança utilizadas no fluxo de recuperação de conta. Essa é uma informação valiosa para um atacante em um ataque de engenharia social ou força bruta de recuperação de conta.

6.6. Parâmetros de busca e filtro

O endpoint /rest/products/search recebe o parâmetro q diretamente na query string, sendo um candidato clássico para testes de injeção (SQL/NoSQL Injection), característica conhecida do Juice Shop.

7. Conclusão

O mapeamento de superfície de ataque realizado na aplicação OWASP Juice Shop, por meio da interceptação de tráfego com o Burp Suite, permitiu identificar 12 endpoints distintos (de um total superior a 20 mapeados), cobrindo os métodos GET e POST, com parâmetros controlados pelo cliente em path, query string e corpo da requisição.

A análise evidenciou múltiplos pontos de atenção do ponto de vista de segurança ofensiva, com destaque para exposição de dados pessoais e de pagamento, possíveis vetores de IDOR/Mass Assignment, e endpoints públicos retornando informação sensível sem autenticação. Todos esses são pontos de partida relevantes para as próximas fases de um teste de intrusão (enumeração aprofundada, exploração de autenticação/autorização e testes de injeção).